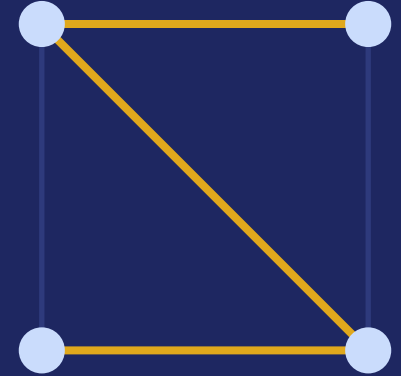


Metodo delle Correnti di Maglia

Implementazione in C++ per la risoluzione di circuiti elettrici

Riccardo Belvisotti

Politecnico di Torino · Matematica per l'Ingegneria · Programmazione e Calcolo Scientifico



Obiettivo del progetto

Implementare il metodo delle correnti di maglia per risolvere circuiti composti da resistori e generatori ideali di tensione.

Cosa calcola

- la corrente che scorre in ogni resistore
- la caduta di tensione ai suoi capi, dalla legge di Ohm $V = R I$
- traducendo il circuito in un sistema lineare

Ipotesi semplificative

- solo resistori e generatori ideali di tensione
- nessun componente in parallelo
- la netlist descrive un unico circuito (grafo connesso)

Scomposizione del progetto



`netlist -> grafo -> maglie -> B, R, v -> A i = v -> tensioni`

Il cuore del progetto sono i due passaggi centrali: implementare gli algoritmi per trovare le maglie e riempire correttamente le matrici tenendo conto dei segni.

Lettura della netlist

Funzione lettura_netlist

- legge la netlist riga per riga con `std::getline`
- ignora righe vuote o malformate senza fermarsi
- ignora eventuali componenti inesistenti (es. A1)
- salva i componenti in una map (chiave R1, V2...) e ne conserva l'ordine in un vector

```
if (linea.empty() || linea.find_first_not_of(" \t\r\n") == std::string::npos)
    continue;

std::istringstream iss(linea);

//caso in cui la riga non è vuota ma il formato è sbagliato
if (!(iss >> chiave >> val.modulo >> val.n1 >> val.n2)) {
    std::cerr << "Attenzione, riga malformata ignorata: '" << linea << "'\n";
    continue;
}

//caso in cui il nome del componente non comincia per 'R' o per 'V'
if (chiave[0] != 'R' && chiave[0] != 'V') {
    std::cerr << "Attenzione, componente sconosciuto '" << chiave << "'
    ignorato nella riga: '" << linea << "'\n";
    continue;
}
```

Esempio netlist:

```
R1 10 1 2
R2 20 2 3
R3 30 2 4
V1 10 1 3
V2 20 4 3
```

Possibili errori da gestire:

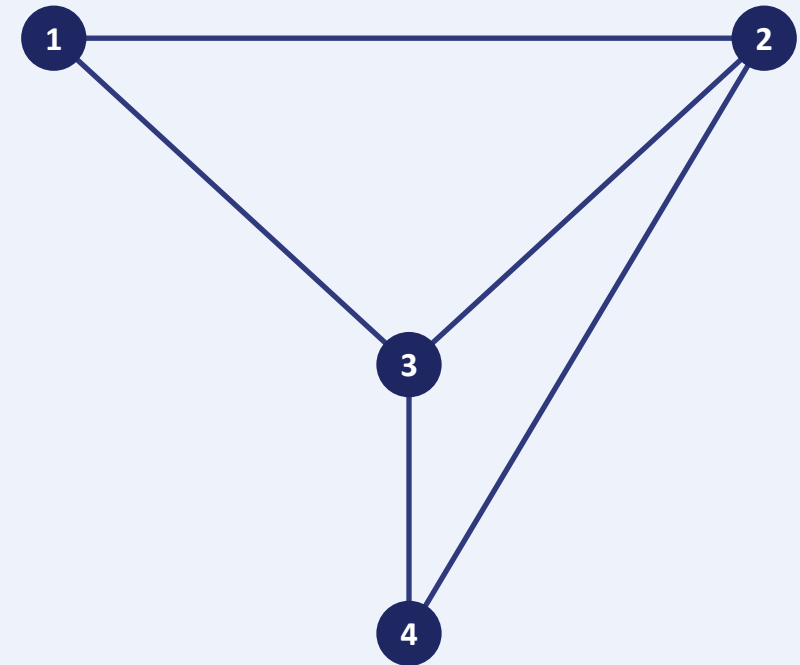
- Componente inesistente: A1 10 1 2
- Spaziatura errata: R2 20 2 3
- Riga vuota:
- Dato mancante: V1 10 1

Il circuito diventa grafo

- riutilizzo di buona parte del codice dell'esercitazione 8 (implementazione di `undirected_edge` e `undirected_graph`)
- nodi = giunzioni, archi = componenti
- arco sempre percorso dal nodo minore al maggiore (imposto dal costruttore di `undirected_edge`)
- overload degli operatori
- lista di adiacenza (`std::map`) e vettore degli archi (`std::vector`) ordinati secondo l'ordine lessicografico

```
//Costruttore user-defined che ordina i nodi in modo che l'arco sia sempre
percorso dal nodo di indice minore a quello maggiore
undirected_edge(I a, I b) {
    if (a <= b) {
        start = a;
        end = b;
    } else {
        start = b;
        end = a;
    }
}
```

Esempio: 4 nodi, 5 archi

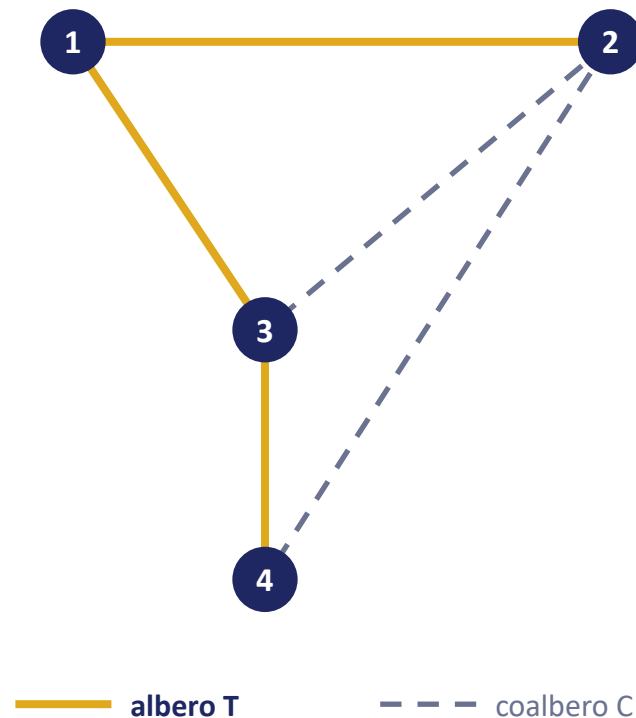


archi lessicografici: (1,2) (1,3) (2,3) (2,4) (3,4)

Implementazione degli algoritmi di visita

Una parte consistente del progetto è stata l'implementazione degli algoritmi di visita del grafo, in cui ho riutilizzato parte del codice dell'esercitazione 9. Per prima cosa ho definito i contenitori FIFO e LIFO e l'algoritmo `visite_grafi`, che fa BFS o DFS a seconda del contenitore utilizzato.

Definendo uno stack (LIFO), si effettua una visita **DFS** del grafo e si ottiene un **albero di ricoprimento** T. Il **coalbero** e' cio' che resta: $C = G \setminus T$.



Il conteggio e' esatto

- un albero su $|V|$ nodi ha $|V| - 1$ archi
- quindi il coalbero ha $|E| - |V| + 1$ archi
- ed e' esattamente il numero di maglie indipendenti del circuito!!!

Nell'esempio: $5 - 4 + 1 = 2$ maglie

DFS e coalbero per trovare i cicli fondamentali

Calcolo dei cicli

Ogni arco del coalbero, reinserito nell'albero, chiude una e una sola maglia: l'arco stesso piu' l'unico cammino tra i suoi estremi in T.

Per ogni arco (u, v) del coalbero, esiste un unico cammino da u a v nell'albero T

La funzione ricorsiva trova_percorso lo trova con backtracking: se un ramo non porta a destinazione, torna indietro (pop_back). I cicli trovati sono corretti, ma non necessariamente minimi (si aggiunge il nodo di partenza u alla fine del percorso per chiudere il ciclo)

arco di coalbero $(2,3)$

cammino in T:

2 -> 1 -> 3

ciclo:

2 -> 1 -> 3 -> 2

lunghezza 3

arco di coalbero $(2,4)$

cammino in T:

2 -> 1 -> 3 -> 4

ciclo:

2 -> 1 -> 3 -> 4 -> 2

lunghezza 4

Riepilogo passaggi implementativi:

- Utilizzo il DFS (contenitore LIFO) per trovare l'albero di ricoprimento T
- Calcolo il coalbero $C = G \setminus T$
- Per ogni arco (u,v) del coalbero uso la funzione trova_percorso per cercare il percorso da u a v in T
- Chiudo il percorso con il nodo u per formare il ciclo
- Costruisco il vettore delle maglie con la funzione cicli_fondamentali_dfs

De Pina: cicli minimi

Un secondo algoritmo, alternativo, che cerca la base di cicli di lunghezza minima. Anche qui il primo passo è trovare l'albero di ricoprimento DFS e il relativo Coalbero, ma poi si procede in modo diverso e i risultati ottenuti possono essere diversi.

DFS + coalbero

2 -> 1 -> 3 -> 5 -> 2

lunghezza 4

De Pina

2 -> 5 -> 3 -> 2

lunghezza 3 (minima)

stesso circuito, stessa maglia trovata piu' corta

Funzione algoritmo_depina:

1

Testimoni su F2

Si inizializzano k (= num archi coalbero) vettori booleani S_i (chiamati testimoni); a ogni passo si cerca il ciclo minimo non ortogonale al testimone corrente (ossia tale che il prodotto scalare mod 2 sia uguale a 0)

↓
prodotto_mod2

2

Grafo di lifting e BFS per trovare il ciclo minimo

Ogni nodo del grafo viene duplicato, si inseriscono gli archi nel grafo di lifting e si cerca il cammino minimo tra v^+ e v^- , utilizzando BFS. Infine si sceglie uno dei vettori di incidenza con il minore numero di 1

↓
vettore_incidenza
bfs_cammino_minimo
ciclo_minimo_con_parita

3

Ortogonalizzazione

Utilizzando la differenza simmetrica (XOR) tra i testimoni si mantengono i cicli indipendenti

↓
differenza_simmetrica

Costruzione del sistema lineare

La funzione `calcola_sistema`, che riceve in input la map che rappresenta il circuito e il vettore `ordine_componenti`, costruisce le matrici necessarie per il sistema lineare.

R

matrice diagonale (num_resistori x num_resistori)

Itero su tutti i resistori, inserendo i rispettivi valori sulla diagonale di R

B

matrice di incidenza (num_resistori x num_maglie)

Inserisco +1 se la maglia percorre il resistore nel verso canonico (ovvero dal nodo minore al maggiore), -1 se in verso opposto, 0 se non lo tocca

V

termine noto (vettore delle tensioni dei generatori)

Se il generatore è attraversato da - a + la tensione è positiva, da + a - tensione negativa

NOTA:

Nella netlist, l'ordine di inserimento dei nodi è importante per i generatori di tensione, mentre è del tutto ininfluente per i resistori.

```
if (key[0] == 'R') {
    R(i_res, i_res) = valori.modulo;
    I r_start = std::min(n1, n2);
    I r_end = std::max(n1, n2);
    for (int j = 0; j < num_maglie; j++) {
        if (archi_maglie[j].contains({r_start, r_end})) {
            B(i_res, j) = 1; }
        else if (archi_maglie[j].contains({r_end, r_start})) {
            B(i_res, j) = -1; }
    } }
}
```

```
else if (key[0] == 'V') {
    for (int j = 0; j < num_maglie; j++)
        if (archi_maglie[j].contains({n2, n1})) {
            v(j) += valori.modulo;
        }
        else if (archi_maglie[j].contains({n1, n2})) {
            v(j) -= valori.modulo;
        }
    }
}
```

Gradiente coniugato e stampa risultati

$$A = B^T R B$$

1

Simmetrica

$$A^T = (B^T R B)^T = B^T R^T B = B^T R B = A$$

perché R é diagonale

2

Definita positiva

$$x^T A x = (Bx)^T R (Bx) = \sum_i R_{ii} (Bx)_i^2 > 0$$

Dalla risoluzione del sistema ottengo le correnti di maglia i, quindi calcolo:

- correnti sui resistori = B_i ;
- tensioni sui resistori = $R B_i$.

Infine, procedo con la stampa dei risultati ----->

Perche' e' il metodo giusto

- A simmetrica definita positiva (SPD): il gradiente coniugato e' il metodo d'elezione (riutilizzo codice esercitazione 10)
- converge in al piu' n iterazioni (n = numero di maglie)
- non richiede di fattorizzare A
- it_max = num_maglie + it_extra, con un margine per l'aritmetica in virgola mobile

```
void stampa(const std::vector<std::string>& ordine_componenti, const Eigen::VectorXd&
tensioni, const Eigen::VectorXd& correnti) {
    int i_res=0;
    std::cout.precision(4); //imposto le cifre significative
    for (const std::string& key : ordine_componenti) {
        if (key[0] == 'R') {
            std::cout << key << ": V = " << tensioni(i_res) << " volts, I = " <<
correnti(i_res) << " amps.\n";
            i_res++;
        } } }
```

Verifica e strumenti utilizzati

Verifica dei risultati

- output confrontato con i risultati attesi della consegna: coincidono
- DFS e De Pina danno maglie diverse ma le stesse tensioni fisiche
- le correnti di maglia sono fittizie (cambiano in base alla base di cicli scelta); le grandezze sui resistori sono fisiche e non dipendono dall'algoritmo

R1: $V = 8 \text{ volts}$, $I = 2 \text{ amps}$.
R2: $V = 22 \text{ volts}$, $I = 2.2 \text{ amps}$.
R3: $V = -6 \text{ volts}$, $I = -0.2 \text{ amps}$.
R4: $V = -28 \text{ volts}$, $I = -2.8 \text{ amps}$.
R5: $V = 12 \text{ volts}$, $I = 3 \text{ amps}$.

stessi valori con entrambi gli algoritmi

Uso di strumenti di IA

Progetto sviluppato in autonomia

parser, grafo, visite, DFS + coalbero, costruzione del sistema e gradiente coniugato: scritti da me, riutilizzando il codice delle esercitazioni.

Algoritmo di De Pina

sviluppato con l'assistenza di uno strumento di IA, sulle mie interfacce, e successivamente verificato: risultati confrontati con il metodo DFS e con i valori attesi.

In sintesi

1

Il circuito e' un grafo

netlist -> grafo -> cicli fondamentali -> sistema lineare: un procedimento meccanico.

2

Due strade per le maglie

DFS + coalbero (semplice, cicli non minimi) e De Pina (cicli minimi).

3

I segni sono il punto critico

risolti confrontando coppie ordinate di nodi, senza gestione manuale.

4

La struttura sceglie il solver

$A = B^T R B$ e' SPD per costruzione: il gradiente coniugato e' la scelta naturale.

Grazie per l'attenzione